

CSC Problem Set 03 +X Report

Name: Nour Mahmoud

Email: nhmahmou@ncsu.edu

Section: 001

Collaborators: N/A

Summary:

For my +X implementation, I extended my original submission, which used Simulated Annealing, by implementing a Genetic Algorithm as an alternative optimization approach. Through testing and evaluation, I found that the Genetic Algorithm yielded better results, so I chose to submit it as my final solution. Additionally, I enhanced the visualization tool by introducing interactive controls, including start and stop buttons to manage the simulation and an algorithm selector to switch between Genetic Algorithm and Simulated Annealing. These improvements provide a more dynamic and user friendly way to observe how different optimization strategies perform within the environment.

"+X" Concept:

For my +X implementation, I expanded my original optimization approach by incorporating a Genetic Algorithm alongside Simulated Annealing. My initial implementation focused on Simulated Annealing, a probabilistic technique that tries to find the best local maxima it can find, which can occasionally be the global maxima. While effective, I observed that it struggled with a larger number of tasks. To improve upon this, I implemented a Genetic Algorithm, which mimics natural selection by using mutation, crossover, and selection to evolve solutions over generations. This allowed it to explore the search space more thoroughly and avoid getting stuck in local optima. After testing both methods, I found that the Genetic Algorithm consistently outperformed Simulated Annealing, producing better solutions more reliably. Because of this, I decided to submit the Genetic Algorithm as my final implementation.

In addition to improving the optimization algorithm, I also enhanced the visualization to make testing and comparison more interactive. I implemented a user interface that allows users to start and stop, as well as dynamically switch between the Genetic Algorithm and Simulated Annealing. This required modifying the environment class to properly reset configurations and ensure accurate score tracking when restarting simulations. These features provide a more intuitive and flexible way to observe how each algorithm performs, making it easier to analyze their behavior in real-time.

Technical Implementation of +X:

For my +X, I implemented both Genetic Algorithm (GA) and Simulated Annealing (SA) as optimization methods to solve the linear assignment problem. To allow users to switch between

these two algorithms dynamically, I introduced an algorithm selection feature in the visualization. I added a combo box in the GUI that lets the user choose between both algorithms before starting the simulation [1]. When an algorithm is selected, the environment resets, and the correct algorithm is assigned internally. To support this, I modified my Environment class to include a setter for the algorithm type and ensured that switching the algorithm also resets the current configuration, best configuration, and previous configurations. For example, in the VisualizeSimulation class, when the combo box selection changes, I call my **resetSimulation()** method, which stops any ongoing simulation, reloads the environment, and applies the new algorithm choice. In the Environment class, I added a **setAlgorithm(String algorithm)** method, amongst other helper methods, to ensure that the ConfigurationSolver is updated accordingly. Without this setter, the simulation would not recognize the new algorithm even after switching. Additionally, I implemented reset functions inside the Environment class to clear previous configurations and properly reconstruct the search process when the simulation is restarted. The start and stop functionality remained mostly unchanged from previous problem sets, where I used a timer to control the execution of the simulation. The user can pause and resume the process using the start and stop buttons, and the visualization updates accordingly.

```
//combo box for selecting algorithm
algorithmSelector.addActionListener(e -> {
    String selected = (String) algorithmSelector.getSelectedItem();
    resetSimulation();
    env.setAlgorithm(selected.equals("Genetic Algorithm") ? "GA" : "SA");
    System.out.println("Algorithm changed to: " + selected);
});
```

```
//reset Simulation when algorithm changes
private void resetSimulation() {
    envPanel.stopSimulation();
    int[][] map = InputManager.LoadMap(mapFile);
    env = new Environment(map);
    env.clearConfigurations();
    envPanel.setEnvironment(env);
    envPanel.resetSimulation();
    scorePanel.resetScore();
}
```

```
public void setCurrentConfiguration(int[] newConfig) {
    this.currentConfiguration = newConfig;
}

public void clearConfigurations() {
    this.currentConfiguration = new int[this.numWorkers];
    this.solver.resetSolver();
}

public void setAlgorithm(String algorithm) {
    resetSolver();
    this.solver.setAlgorithm(algorithm);
}

//reconstructs the config solver
private void resetSolver() {
    this.solver = new ConfigurationSolver(this);
    this.previousConfigurations = new PriorityQueue<>();
    this.currentConfiguration = new int[this.numWorkers];
}
```

Each algorithm takes a different approach to finding an optimal solution, and implementing both allowed me to compare their effectiveness in solving the problem. Simulated Annealing starts with an initial configuration and makes small changes, sometimes accepting worse solutions to escape local optima. The probability of accepting worse solutions decreases over time as the temperature parameter cools down, reducing randomness and guiding the search toward better solutions.

In my Simulated Annealing implementation, I used a cooling schedule to control how the temperature decreases, starting at 100. The core of the algorithm involves modifying one task assignment at a time, calculating the new score, and deciding whether to keep the change based on fitness and probability. If the new configuration has a better score, it is always accepted. If it's worse, it is accepted with a probability using the temperature value. This allows escaping local optima. For neighbor selection, I introduced a probabilistic swapping mechanism, where 75% of the time, the algorithm swaps two random workers, making localized adjustments, while 25% of the time, it swaps three workers, allowing for broader exploration.

This balances small refinements with occasional larger changes to prevent getting stuck at local maximas. I used a linear cooling schedule, decreasing the temperature by 0.1 per iteration, which provides a steady reduction in randomness, allowing broad exploration at high temperatures. This approach ensures a controlled decrease in randomness, preventing the search from converging too quickly. However, despite tuning these parameters, SA sometimes struggled to consistently find the global best configuration, which led me to explore Genetic Algorithm (GA) as an alternative.

```
//this method lets us swap workers in each iteration
//it maintains knowledge of prev sols
//it takes the configuration array and swaps 2 or 3 random elements from that
private int[] randomNeighbor(int[] state) {
    int[] neighbor = state.clone();
    Random rand = new Random();

    //swap 2 workers 75% of the time
    if (Math.random() < 0.75) {
        int i = rand.nextInt(neighbor.length);
        int j = rand.nextInt(neighbor.length);
        while (i == j) {
            j = rand.nextInt(neighbor.length);
        }
        int temp = neighbor[i];
        neighbor[i] = neighbor[j];
        neighbor[j] = temp;
    } else {
        int i = rand.nextInt(neighbor.length);
        int j = rand.nextInt(neighbor.length);
        int k = rand.nextInt(neighbor.length);
        while (j == i) {
            j = rand.nextInt(neighbor.length);
        }
        while (k == i || k == j) {
            k = rand.nextInt(neighbor.length);
        }
        int temp = neighbor[i];
        neighbor[i] = neighbor[j];
        neighbor[j] = neighbor[k];
        neighbor[k] = temp;
    }
    return neighbor;
}

public int[] updateSearch () {
    //update the temperature
    temperature = schedule();

    //stopping condition: if temperature is 0 or less
    if (temperature <= 0) {
        return bestConfiguration;
    }

    //generate a neighbor
    int[] candidate = randomNeighbor(configuration);

    //compute energy and delta E
    double current = env.calcScore(configuration);
    double next = env.calcScore(candidate);
    double dE = next - current;

    //accept candidate based on fitness and probability
    if (dE > 0) {
        configuration = candidate;
    } else {
        if (Math.random() < probability(dE, temperature)){
            configuration = candidate;
        }
    }

    //track the best observed solution
    if (env.calcScore(configuration) > env.calcScore(bestConfiguration)) {
        bestConfiguration = configuration.clone();
    }

    return configuration;
}

// this method updates the temperature through linear decay intervals
private double schedule() {
    return temperature - 0.1;
}
```

For my Genetic Algorithm implementation, I took an evolutionary approach where multiple configurations evolve over time. Instead of focusing on a single configuration like SA, GA maintains a population of solutions and generates new ones based on survival of the fittest. I started by initializing a random population of configurations. The fitness function evaluated each configuration based on the total task assignment score. To evolve better solutions, I implemented selection, crossover, and mutation. **Selection** was done through the comparison of multiple solutions, and the best two were chosen to reproduce and pass their characteristics to the next generation. Occasionally though, 20% of the time, the worst and the best configurations are selected. This ensures that we do not get stuck with the same population. For crossover or **reproduction**, I implemented a crossover, where two parent configurations were combined by swapping a portion of their task assignments, generating a child solution. To maintain diversity, I included a mutation step, where random task assignments were swapped within a configuration to introduce variation. Mutation ensured that the search did not stagnate

and could explore a broader range of solutions.

```
private int[][] selection() {
    Random rand = new Random();
    if (rand.nextDouble() < 0.8) { //80% chance: pick best 2
        return new int[][] {population.get(0), population.get(1)};
    } else { //20% chance: pick a weakest & strongest
        return new int[][] {population.get(0), population.get(population.size() - 1)};
    }
}
```

```
private int[] reproduce(int[] parent1, int[] parent2) {
    //initialize a child with the same number of elements as the parents
    int[] child = new int[parent1.length];
    //used stackoverflow to get this so I dont have to use a loop
    Arrays.fill(child, -1);

    Random rand = new Random();

    //pick a random start that would be from the start of the array up till half of the array
    int start = rand.nextInt(parent1.length / 2);

    //pick an end that would be after the start and before the last index
    int end = start + rand.nextInt(parent1.length - start);

    //copy a random slice from parent1 according to the random choices of the start and end
    for (int i = start; i < end; i++) {
        child[i] = parent1[i];
    }

    //fill remaining slots from parent2 (ensuring no duplicates)
    // basically make sure that if parent 1 is 1 2 3 4 5
    // and parent two is 5 2 3 4 1, this ensures that if the selection is from 0 to 2 from first parent
    // it does not repeat the elements from the second, so it does not do 1 2 3 4 1 --> 1 is repeated...
    // this ensures that does not happen
    int index = 0;
    for (int num : parent2) {
        if (!contains(child, num)) {
            while (child[index] != -1) {
                index++;
            }
            child[index] = num;
        }
    }
    return child;
}
```

```
//this is called 80% of the time and swaps two randomly chosen elements of the array
private void mutate(int[] individual) {
    Random rand = new Random();
    int i = rand.nextInt(individual.length);
    int j = rand.nextInt(individual.length);
    while (i == j) {
        j = rand.nextInt(individual.length);
    }

    int temp = individual[i];
    individual[i] = individual[j];
    individual[j] = temp;
}
```

In conclusion, implementing both Genetic Algorithm (GA) and Simulated Annealing (SA) allowed me to compare physics based algorithms and evolutionary based algorithms for solving the linear assignment problem. SA enabled broad exploration but was inconsistent, while GA's selection, crossover, and mutation produced more reliable results. Adding an algorithm selection feature and ensuring proper environment resets made it easier to compare their performance. This implementation deepened my understanding of optimization techniques.

Evaluation and Results:

The results of adding Genetic Algorithm and Simulated Annealing as optimization methods highlight a clear advantage of GA, particularly for larger problem instances. In smaller cases, such as the 5x5 task worker assignments, both algorithms performed similarly, achieving the optimal scores of 134 and 450, suggesting that for small scale problems, SA is sufficient. However, as the problem size increased to 9x9, GA consistently outperformed SA. For example, GA reached a score of 804, which is the best possible solution, whereas SA struggled, with its highest score being 706. In other cases, GA achieved scores of 725 and 754, while SA lagged behind at 693 and 681, showing a noticeable difference in effectiveness.

	Simulated Annealing Score	Genetic Algorithm Score	Highest Possible Score
Input 1	134	134	134
Input 2	450	450	450
Input 3	693	804	804
Input 4	681	754	754
Input 5	706	747	785

The key takeaway from these results is that while SA provides a fast and effective method for small problems, its reliance on single solution exploration with probability makes it more prone to settling into local optima. GA, on the other hand, benefits from maintaining a diverse population, crossover, and mutation mechanisms, which allow it to escape local optima and find better global solutions. This is particularly relevant in real world applications where small improvements in task assignments can translate into significant productivity and cost savings.

The visualization component further strengthens these findings by allowing users to dynamically switch between GA and SA, providing an intuitive way to compare their performance. Through real-time updates, users can observe how each algorithm progresses and converges to different solutions, reinforcing the advantages of GA in larger and more complex scenarios. The ability to reset and restart simulations ensures fair comparisons. The results demonstrate that when precision and optimality are critical, GA is the superior choice for task allocation problems.



Works Referenced:

1. Oracle, "How to Use Combo Boxes (The Java™ Tutorials)," Oracle Java Documentation, 2023. [Online]. Available:
<https://docs.oracle.com/javase/tutorial/uiswing/components/combobox.html>.
2. Stack Overflow, "Any shortcut to initialize all array elements to zero?" Stack Overflow, 2010. [Online]. Available:
<https://stackoverflow.com/questions/2154251/any-shortcut-to-initialize-all-array-elements-to-zero>.
3. A. Gaweda, "C06 - Optimization and Meta-Heuristics," CSC 411, North Carolina State University, Lecture 6, Feb. 2025. [Lecture slides]